Eviction & Sizing

systemdesignschool.io/fundamentals/cache-eviction-sizing

TTL removes entries that have outlived their freshness. But what happens when the cache fills up and no entries have expired yet? New data needs space. The eviction policy decides which existing entry to sacrifice.

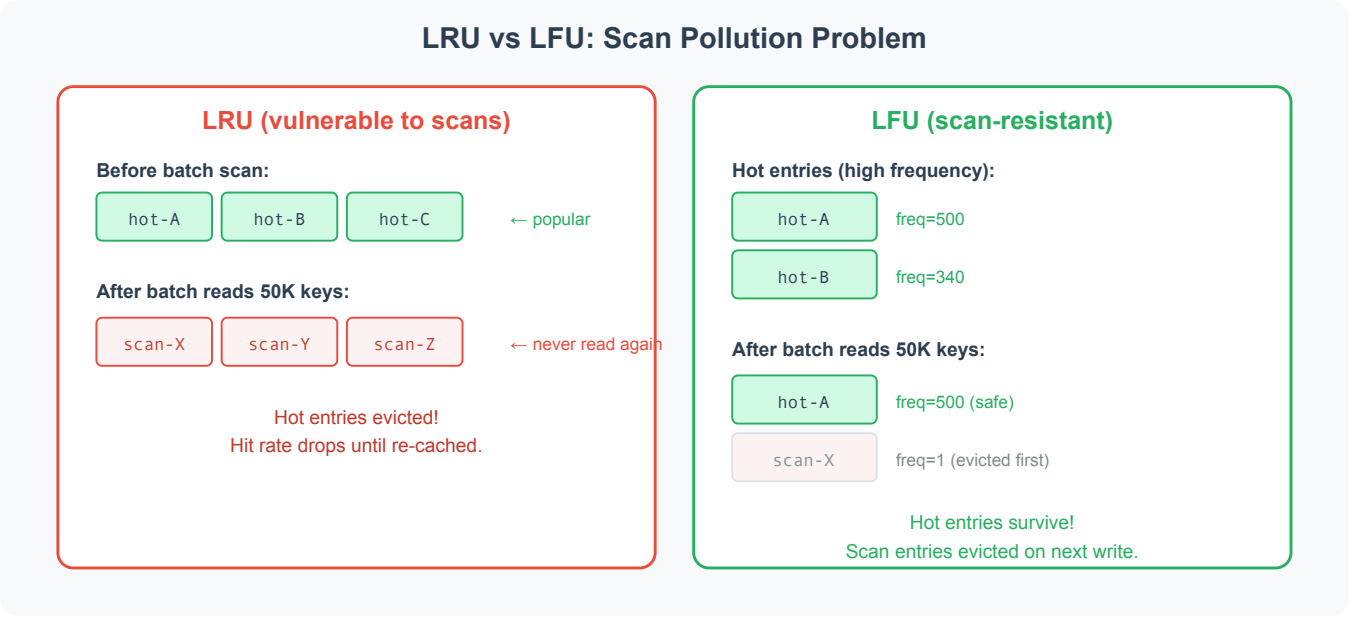
Why Eviction Matters

Memory is finite and expensive. A Redis instance with 16 GB of RAM can hold roughly 100–200 million small key-value pairs (depending on value size). When the cache reaches capacity, every new write must evict an existing entry. The eviction policy determines whether the sacrificed entry was useful (causing a future miss) or dead weight (never to be read again).

A good eviction policy maximizes hit rate—the proportion of reads served from cache rather than the database. A bad policy evicts entries that will be needed again soon, turning cache writes into wasted effort.

LRU: Least Recently Used

LRU evicts the entry that has not been accessed for the longest time. The assumption: entries accessed recently are likely to be accessed again soon. Entries untouched for a long period are probably cold.



LRU works well for workloads with **temporal locality**—when recent activity predicts near-future activity. Web sessions, user profiles, and page content follow this pattern. A user browsing a product category will likely revisit the same products within minutes.

Weakness. A scan operation (one-time traversal of many keys) pollutes the cache. A batch job that reads 50,000 user records once pushes out the frequently-accessed hot entries. After the scan, the cache can fill with entries unlikely to be read again. Hit rate drops until the hot entries are re-cached through natural traffic.

Redis supports **approximated LRU** (via `allkeys-lru` or `volatile-lru` policies). Rather than maintaining a sorted list of all keys by access time (expensive in memory), Redis samples a small number of keys and evicts the least recently used among the sample. This trades perfect accuracy for low overhead. Note: Redis defaults to `noeviction`—you must configure an eviction policy and set `maxmemory` for eviction to occur.

LFU: Least Frequently Used

LFU evicts the entry with the fewest accesses. It counts how often each entry is read and removes the one with the lowest count. The assumption: entries accessed many times are important; entries accessed rarely are disposable.

LFU resists the scan problem that plagues LRU. A batch job that reads 50,000 keys once gives each a frequency count of 1. The hot entries with counts of 500+ are safe from eviction.

Weakness. LFU has a cold-start problem. A newly cached entry starts with a count of 1. If the cache is full of entries with high counts, the new entry gets evicted immediately—even if it's about to become the most popular key. Some LFU implementations solve this with **frequency decay**: counts decrease over time, so historically popular but now-cold entries gradually lose their protection.

Redis implements LFU with a logarithmic counter and decay factor (`lfu-log-factor` and `lfu-decay-time` configuration). The logarithmic counter means the count saturates quickly—the difference between 100 and 1000 accesses is small, preventing older entries from being permanently immune.

When to Use Which

Workload Pattern	Best Policy	Reasoning
User sessions, page views	LRU	Recent access predicts near-future access
Product catalog, trending items	LFU	Popular items should survive regardless of recency
Mixed with batch jobs	LFU	Resists scan pollution
Streaming data, event logs	Short TTL	One-time-read data benefits little from caching; use short TTL if caching for brief replays
Unknown or uniform access	LRU	Simpler, good default

For most web applications, **LRU is the safer default**. Switch to LFU when you observe scan pollution (hit rate drops during batch operations) or when a small set of keys accounts for the majority of traffic.

Memory Math: Sizing the Cache

How much memory does the cache need? Too small means frequent evictions and low hit rate. Too large means paying for unused RAM.

The calculation starts with the working set—the number of distinct keys accessed within one TTL window.

Step 1: Estimate the number of hot keys. If the application serves 10,000 unique product pages per hour and the TTL is 15 minutes, the working set is roughly 2,500 keys (one quarter-hour's worth of unique accesses).

Step 2: Estimate average entry size. Each cached entry includes the key, value, and Redis metadata overhead (~70–100 bytes per key in Redis). A product page cache entry might be:

- Key: 30 bytes (`product:v1:detail:12345`)
- Value: 2 KB (serialized JSON)
- Overhead: 80 bytes
- Total: ~2.1 KB per entry

Step 3: Calculate total memory. 2,500 keys × 2.1 KB = ~5.3 MB. Add 20–30% headroom for fragmentation and transient spikes: ~7 MB.

For larger systems, the same formula scales. A cache serving 1 million unique keys at 2 KB each needs ~2 GB plus headroom—roughly 2.5–3 GB allocated.

The 80/20 heuristic. Many read-heavy workloads are skewed—often approximated by a rule where 20% of keys serve 80% of traffic (Zipf-like distribution). Confirm your workload's skew from access logs before sizing. Caching only the hot 20% captures most of the benefit. If the total dataset is 10 million keys, caching 2 million keys (the hot set) may achieve a 90%+ hit rate. This means the cache can be much smaller than the total dataset.

Monitoring Eviction Health

Two metrics signal whether the cache is correctly sized:

Hit rate. The percentage of reads served from cache. Below 90% for a well-tuned cache suggests the working set exceeds cache capacity. Above 99% may mean the cache is oversized (paying for unused memory).

Eviction rate. How many entries Redis evicts per second. A steady eviction rate at maximum memory means the cache is at capacity—normal behavior. A spike in evictions correlates with batch operations or traffic bursts that push out hot entries.

If the eviction rate is high and hit rate is dropping, the cache needs more memory or a better eviction policy. If evictions are rare and hit rate is consistently above 99%, the cache may be over-provisioned.

From here, the discussion moves from single-node caching to distributed caching—how to shard data across multiple cache nodes when one machine can't hold the entire working set.